



ACTIVIDAD ACADEMICA

Espacio de Estados y Búsqueda Inteligente

Gerson Alvarado Alvarado

Jhoel Alvaro Cruz Zurita

Yery Eguez Cortez

Adriana Rodríguez Sotelo

Douglas Flor Hernandez

Ingeniería de Sistemas

Ing. Requena Llorentty Jimmy Nataniel

Santa Cruz de la Sierra - Bolivia

julio de 2026

INDICE

1. Resumen ejecutivo.....	3
---------------------------	---

2.	Selección del caso.....	3
2.1.	Caso elegido: Ojo Camba — la ruta del técnico.....	3
2.2.	Contexto real y anclaje geográfico.....	4
2.3.	Vínculo con los Objetivos de Desarrollo Sostenible (ODS).....	4
3.	Fundamentación.....	4
3.1	Antecedentes históricos de la IA relevantes al caso.....	4
3.2.	Justificación del enfoque: IA simbólica (búsqueda), no estadística.....	5
4.	Formalización del espacio de estados.....	6
4.1.	Estado inicial, estado meta, operadores y función de costo.....	6
4.2.	El espacio de estados completo (grafo del caso).....	7
4.3.	Caracterización del entorno del problema.....	8
5.	Paso 4 · Implementación.....	9
5.1.	Cómo usarlo en Google Colab.....	9
5.2.	Definición del espacio de estados.....	9
5.3.	BFS — explora nivel por nivel.....	10
5.4.	DFS — profundiza y se queda con la primera ruta completa.....	11
5.5.	Backtracking — poda por costo y por prioridad de gravedad.....	11
5.6.	Ejecución y comparación.....	12
6.	Análisis comparativo.....	13
6.1.	Metodología.....	13
6.2.	Caso 1 — lote típico de 5 reportes (Base → visitar R101...R105).....	13
6.3.	Caso 2 — contraprueba con orden de exploración adverso.....	14
6.4.	El “precio de la urgencia”.....	14
6.5.	Recomendación final fundamentada.....	15
7.	Auditoría de IA.....	15
7.1.	Auditoría crítica destacada.....	16
8.	Conclusiones.....	17
9.	Referencias.....	17

1. Resumen ejecutivo

Este informe formaliza y resuelve, como espacio de estados, un problema real del sistema Ojo Camba: una vez que el Back Office aprueba un lote de reportes ciudadanos (baches, luminarias, canales, emergencias, etc.), un técnico de campo debe visitarlos todos partiendo de una base, en el orden que minimice la distancia recorrida sin dejar de atender primero los reportes más urgentes. Se formaliza el problema como espacio de estados (estado, meta, operadores, función de costo), se caracteriza su entorno, y se implementan y comparan tres algoritmos de búsqueda simbólica —BFS, DFS y Backtracking— sobre dos instancias del problema: un caso base y una contraprueba diseñada para exponer los límites de BFS y DFS. Los resultados, obtenidos ejecutando el código real (no simulados), muestran que solo backtracking con poda por costo y por prioridad garantiza simultáneamente el costo mínimo y el cumplimiento de la regla de negocio de atención por gravedad.

2. Selección del caso

2.1. Caso elegido: Ojo Camba — la ruta del técnico

Organización: Ojo Camba, plataforma ciudadana de reporte y seguimiento de incidencias urbanas en Santa Cruz de la Sierra, construida como monorepo con microservicios NestJS (ms-auth, ms-register, ms-admin, ms-gamify), dos API Gateways y cuatro aplicaciones cliente (app-reportes para ciudadanos, app-backoffice, app-tecnico y app-status), sobre PostgreSQL 16 con extensión PostGIS.

Problema real: cuando el Back Office aprueba un lote de reportes (estado Aceptado, agrupados como Casos de Obra), un técnico debe visitarlos en persona para validarlos en campo (transición Aceptado → ValidacionEnCampo). Hoy ese orden de visita no está optimizado: se decide de forma manual, sin considerar la distancia real ni la columna gravedad (Emergencia, Alta, Media, Baja) de cada reporte. Esto genera tres consecuencias medibles: (1) recorridos más largos de lo necesario, (2) emergencias que pueden quedar relegadas si el técnico visita primero lo más cercano, y (3) imposibilidad de estimar con criterio técnico cuánto tardará en cerrarse un lote.

Rol asumido: el grupo actúa como equipo técnico del propio proyecto y formaliza el problema de ruteo del técnico como un espacio de estados, comparando algoritmos de búsqueda para recomendar, con criterio técnico, el orden de visita de menor costo que respete la prioridad de gravedad.

2.2. Contexto real y anclaje geográfico

Los nodos del espacio de estados corresponden a la Base del técnico y a coordenadas (lat, lng) reales del tipo almacenado en la tabla reportes de Ojo Camba (que además guarda índices H3 en resoluciones 8, 11 y 13 para indexación espacial). En producción, el grafo puede poblarse con la red vial real de Santa Cruz de la Sierra vía OpenStreetMap + OSMnx (herramientas gratuitas), mapeando cada reporte a su nodo vial más cercano. Para esta actividad, siguiendo el mismo criterio que permite la guía institucional cuando no se dispone de Overpass API en el momento de trabajar, se usan distancias estimadas por el equipo entre 4-5 reportes de ejemplo y la Base, documentadas explícitamente como no oficiales (bitácora, Paso 6).

2.3. Vínculo con los Objetivos de Desarrollo Sostenible (ODS)

El caso se relaciona con el ODS 9 (Industria, Innovación e Infraestructura), al aplicar búsqueda simbólica para optimizar la operación de una infraestructura de gestión urbana ciudadana, y con el ODS 11 (Ciudades y Comunidades Sostenibles), porque reducir los metros recorridos por el técnico reduce combustible, tiempo de respuesta ante emergencias urbanas y congestión vehicular.

3. Fundamentación

3.1 Antecedentes históricos de la IA relevantes al caso

- Conferencia de Dartmouth (1956): John McCarthy acuña el término “inteligencia artificial” y funda la hipótesis simbólica: la inteligencia puede modelarse mediante la manipulación de símbolos y reglas explícitas, sin imitar procesos biológicos.
- Logic Theorist (Newell y Simon, 1956): primer programa en resolver un problema mediante búsqueda paso a paso dentro de un espacio de posibles secuencias de reglas — la misma idea central que aplica este informe.
- General Problem Solver – GPS (Newell y Simon, 1957): generaliza la búsqueda a cualquier problema descriptible como estado inicial, estado meta y operadores mediante análisis medios-fines; es el antecesor conceptual directo de la formalización del Paso 3.

- Shakey the Robot (SRI, 1966–1972): primer robot en combinar percepción del entorno, planificación simbólica (STRIPS) y navegación por búsqueda para desplazarse entre puntos conocidos — antecedente directo de un técnico que decide su propia secuencia de visitas con costo.
- Algoritmo A* (Hart, Nilsson y Raphael, 1968): desarrollado para la planificación de rutas de Shakey; combina el costo real recorrido con una heurística del costo restante. No se implementa aquí porque la consigna exige BFS, DFS y/o backtracking, pero se referencia como evolución natural y como recomendación de trabajo futuro (Sección 5.5).

3.2. Justificación del enfoque: IA simbólica (búsqueda), no estadística

La rúbrica exige justificar por qué el caso corresponde a un enfoque de IA simbólica y no a uno estadístico. La justificación se apoya en tres criterios técnicos:

- El conocimiento del problema es completo y explícito, no se infiere de datos. Se conoce de antemano cada nodo (Base y reportes), cada arista (distancia estimada o real) y la gravedad de cada reporte: esa estructura ya está dada por la tabla reportes de Ojo Camba y no requiere entrenar ningún modelo para descubrirla. La IA estadística se usaría, en cambio, si hubiera que predecir un dato no explícito — por ejemplo, el tiempo de viaje futuro a partir de patrones históricos de tráfico, que sería un problema de regresión, no de búsqueda.
- La solución se obtiene por manipulación de símbolos y reglas, no por ajuste de parámetros. BFS, DFS y backtracking manipulan directamente la representación simbólica del grafo (nodos, aristas, pesos, gravedad) siguiendo reglas exactas y deterministas. No hay función de pérdida que minimizar ni pesos numéricos ajustados a partir de ejemplos históricos, que es el mecanismo central de los modelos conexionistas.
- El problema es determinista y verificable formalmente. Dado el mismo lote de reportes, los mismos operadores producen siempre el mismo resultado, y el costo de una solución puede calcularse de forma exacta sumando los pesos del camino. Esto permite demostrar formalmente que backtracking encuentra la solución óptima

(Sección 5), algo que no podría afirmarse con la misma certeza sobre la salida de un modelo entrenado con datos.

En síntesis, el caso Ojo Camba encaja en el paradigma de “agentes basados en búsqueda sobre un espacio de estados formalmente definido” (Russell y Norvig), heredero directo del Logic Theorist, el GPS y Shakey: el tipo de conocimiento disponible (completo, explícito, determinista) es precisamente el que la búsqueda simbólica está diseñada para explotar — no porque la IA estadística sea inferior, sino porque no es la herramienta adecuada para este tipo de conocimiento.

4. Formalización del espacio de estados

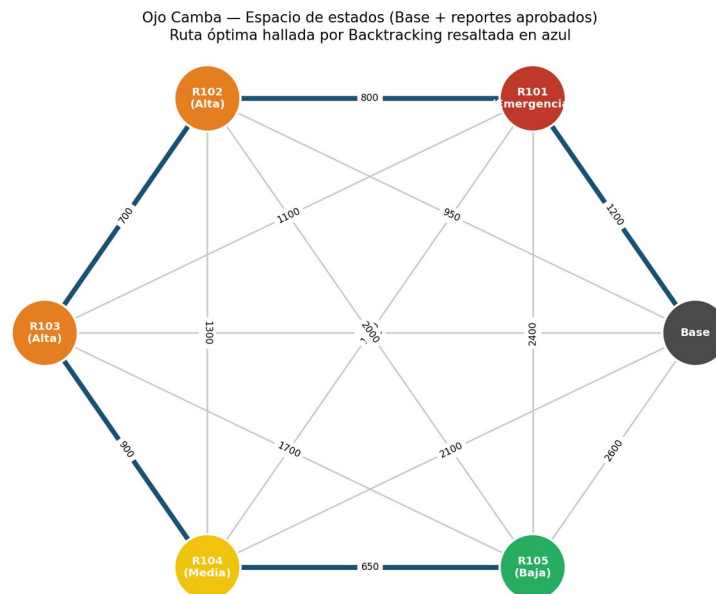
4.1. Estado inicial, estado meta, operadores y función de costo

Elemento	Definición formal para Ojo Camba
Estado	Par (posición actual del técnico, conjunto de reportes ya visitados). No es solo un lugar: incluye qué se ha completado hasta ese momento. Formalmente, un elemento de $\text{Nodos} \times 2^{\text{Reportes}}$.
Estado inicial	$S_0 = (\text{Base}, \emptyset)$: el técnico está en su punto de salida y ningún reporte del lote (grupo de reportes en estado Aceptado, según el enum EstadoReporte de Ojo Camba) ha sido visitado.
Estado meta	$S_g = (\text{cualquier posición}, \text{Reportes})$: todos los reportes aprobados del lote fueron visitados (equivalente a que cada uno transicione a ValidacionEnCampo). La meta es una condición sobre el estado, no un lugar fijo.
Operadores	Visitar(reporte_k): desplaza al técnico desde su posición actual hacia un reporte aún no visitado y lo agrega al conjunto de visitados. Aplicable solo si el reporte no fue visitado antes (sin repetición) — condición que reduce el espacio de un grafo completo a un árbol de permutaciones.
Función de costo	$g(\text{camino}) = \sum \text{distancia}(\text{tramo}_i)$, en metros, sumando cada tramo consecutivo del recorrido. Es aditiva y con pesos estrictamente positivos, condición necesaria para que la poda por cota del backtracking (Sección 4.4) sea válida.
Restricción del dominio	Regla de negocio de Ojo Camba: un reporte no puede visitarse mientras exista, dentro del mismo lote, un reporte de gravedad mayor (Emergencia > Alta > Media > Baja) aún pendiente. Restringe el conjunto de operadores aplicables en cada estado.
Espacio de estados	Grafo completo y no dirigido de 6 nodos (Base + 5 reportes) y 15 aristas ponderadas (Sección 3.2); el árbol de búsqueda inducido por los operadores tiene $5! = 120$ hojas (órdenes de visita posibles) antes de aplicar ninguna restricción.

4.2. El espacio de estados completo (grafo del caso)

El grafo siguiente representa el caso de ejemplo: la Base y 5 reportes aprobados de un mismo lote (R101 Emergencia, R102 y R103 Alta, R104 Media, R105 Baja), con las 15 distancias estimadas en metros entre cada par de nodos (grafo completo y no dirigido). Es exactamente el grafo que se implementa en el código del Paso 4.

Nodo A	Nodo B	Costo	Nodo A	Nodo B	Costo
Base	R101	1200 m	Base	R102	950 m
Base	R103	1500 m	Base	R104	2100 m
Base	R105	2600 m	R101	R102	800 m
R101	R103	1100 m	R101	R104	1900 m
R101	R105	2400 m	R102	R103	700 m
R102	R104	1300 m	R102	R105	2000 m
R103	R104	900 m	R103	R105	1700 m
R104	R105	650 m			



La ruta resaltada en azul es la solución óptima hallada por Backtracking (Paso 5.2): Base → R101 → R102 → R103 → R104 → R105, costo total 4 250 m.

4.3. Caracterización del entorno del problema

Dimensión	Clasificación	Justificación
Observable / no observable	Totalmente observable	La posición, gravedad y distancia de cada reporte se conocen por completo desde la tabla reportes antes de iniciar la búsqueda; el técnico no percibe el entorno de forma incremental ni hay información oculta.
Determinista / estocástico	Determinista	Aplicar <code>Visitar(reporte_k)</code> siempre produce el mismo estado resultante. No se modela variabilidad de tránsito en tiempo real; se declara como límite explícito y como línea de trabajo futuro (costos dinámicos por hora pico).
Estático / dinámico	Estático	El lote de reportes aprobados no cambia mientras el algoritmo busca la solución: no se agregan ni se retiran reportes durante la ejecución de la búsqueda.
Episódico / secuencial	Secuencial	Cada visita modifica qué operadores están disponibles a continuación y el costo acumulado total; el orden de las decisiones importa, no son episodios independientes entre sí.
Tipo de espacio	Discreto	El conjunto de estados (6 nodos × subconjuntos de reportes) y de operadores es finito y enumerable; no hay variables continuas.
Número de agentes	Agente único	Un solo técnico planifica su propia ruta. La extensión a n técnicos corresponde a un Vehicle Routing Problem (Sección 5.5, trabajo futuro).

5. Paso 4 · Implementación

Código en Python, ejecutado y verificado línea por línea antes de incluirse en este informe (la salida real se muestra en el Paso 5). Compatible con Google Colab sin instalar dependencias adicionales (collections, time e itertools son parte de la biblioteca estándar).

5.1. Cómo usarlo en Google Colab

1. Abrir <https://colab.research.google.com> y crear un notebook nuevo.
2. Copiar el contenido de la Sección 4.2 en la primera celda y ejecutarla (Shift+Enter): define el grafo, la gravedad de cada reporte y la función de costo.
3. Copiar las Secciones 4.3, 4.4 y 4.5 en celdas siguientes, en orden, y ejecutarlas una por una.

4. Copiar la Sección 4.6 (ejecución y comparación) en una última celda: imprime la tabla comparativa reproducida en el Paso 5.

5.2. Definición del espacio de estados

```
from collections import deque
import time, itertools

# Grafo completo y no dirigido: Base + 5 reportes aprobados de un lote
grafo = {
    "Base": {"R101": 1200, "R102": 950, "R103": 1500, "R104": 2100, "R105": 2600},
    "R101": {"Base": 1200, "R102": 800, "R103": 1100, "R104": 1900, "R105": 2400},
    "R102": {"Base": 950, "R101": 800, "R103": 700, "R104": 1300, "R105": 2000},
    "R103": {"Base": 1500, "R101": 1100, "R102": 700, "R104": 900, "R105": 1700},
    "R104": {"Base": 2100, "R101": 1900, "R102": 1300, "R103": 900, "R105": 650},
    "R105": {"Base": 2600, "R101": 2400, "R102": 2000, "R103": 1700, "R104": 650},
}

gravedad = {"R101": "Emergencia", "R102": "Alta", "R103": "Alta",
            "R104": "Media", "R105": "Baja"}

orden_gravedad = {"Emergencia": 0, "Alta": 1, "Media": 2, "Baja": 3}

BASE = "Base"

REPORTES = ["R101", "R102", "R103", "R104", "R105"]

def costo_ruta(camino):
    # Funcion de costo: suma las distancias de cada tramo consecutivo
    return sum(grafo[camino[i]][camino[i+1]] for i in range(len(camino)-1))

def respeta_prioridad(camino):
    # Verifica si un camino cumple la regla de gravedad (Emergencia primero)
    vistos = []
    for nodo in camino[1:]:
        rango = orden_gravedad[gravedad[nodo]]
        pendientes_mas_urgentes = [r for r in REPORTES if r not in vistos
                                   and r != nodo and orden_gravedad[gravedad[r]] < rango]
        if pendientes_mas_urgentes:
            return False
        vistos.append(nodo)
```

```
return True
```

5.3. BFS — explora nivel por nivel

Como las rutas completas (visitar los 5 reportes) están todas a la misma profundidad del árbol de búsqueda, BFS debe procesar casi todo el nivel anterior antes de hallar la primera: sirve para evidenciar la explosión combinatoria del problema ($5! = 120$ órdenes posibles).

```
def bfs_visitar_todos(grafo, inicio, reportes):
    nodos_explorados = 0
    frontera = deque([[inicio]])      # cola FIFO de caminos parciales
    while frontera:
        camino = frontera.popleft()   # FIFO: nivel por nivel
        nodos_explorados += 1
        visitados = set(camino[1:])
        if visitados == set(reportes):
            return camino, nodos_explorados
        for vecino in reportes:
            if vecino not in visitados:
                frontera.append(camino + [vecino])
    return None, nodos_explorados
```

5.4. DFS — profundiza y se queda con la primera ruta completa

```
def dfs_visitar_todos(grafo, inicio, reportes):
    nodos_explorados = 0
    pila = [[inicio]]                # pila LIFO de caminos parciales
    while pila:
        camino = pila.pop()           # LIFO: profundiza primero
        nodos_explorados += 1
        visitados = set(camino[1:])
        if visitados == set(reportes):
            return camino, nodos_explorados
        for vecino in reportes:
            if vecino not in visitados:
```

```

        pila.append(camino + [vecino])
    return None, nodos_explorados

```

5.5. Backtracking — poda por costo y por prioridad de gravedad

```

def backtracking_visitar_todos(grafo, inicio, reportes):
    contador = {"nodos": 0}
    mejor = {"camino": None, "costo": float("inf")}

    def prioridad_ok(vecino, visitados):
        rango_v = orden_gravedad[gravedad[vecino]]
        return all(orden_gravedad[gravedad[r]] >= rango_v
                   for r in reportes if r not in visitados and r != vecino)

    def _bt(actual, camino, visitados, costo_acum):
        contador["nodos"] += 1
        if costo_acum >= mejor["costo"]:
            return # PODA por costo
        if visitados == set(reportes):
            if costo_acum < mejor["costo"]:
                mejor["costo"] = costo_acum
                mejor["camino"] = list(camino)
            return
        for vecino, peso in grafo[actual].items():
            if vecino in reportes and vecino not in visitados \
               and prioridad_ok(vecino, visitados): # PODA por prioridad
                visitados.add(vecino); camino.append(vecino)
                _bt(vecino, camino, visitados, costo_acum + peso)
                camino.pop(); visitados.remove(vecino) # deshacer (backtrack)

    _bt(inicio, [inicio], set(), 0)
    return mejor["camino"], contador["nodos"]

```

5.6. Ejecución y comparación

```
resultados = []
for nombre, fn in [("BFS", bfs_visitar_todos), ("DFS", dfs_visitar_todos),
                  ("Backtracking", backtracking_visitar_todos)]:
    t0 = time.perf_counter()
    camino, nodos = fn(grafo, BASE, REPORTES)
    t1 = time.perf_counter()
    costo = costo_ruta(camino)
    resultados.append((nombre, camino, costo, nodos, (t1-t0)*1000, respeta_prioridad(camino)))

print(f"{'Algoritmo':<14}{'Costo(m)':<10}{'Nodos':<8}{'Tiempo(ms)':<12}{'Prioridad':<10}Camino")
for nombre, camino, costo, nodos, tiempo_ms, cumple in resultados:
    print(f"{nombre:<14}{costo:<10}{nodos:<8}{tiempo_ms:<12.4f}{str(cumple):<10}{camino}")
```

6. Análisis comparativo

6.1. Metodología

- Nodos explorados: se incrementa un contador cada vez que un algoritmo procesa (no solo descubre) un estado, siguiendo la forma estándar de medir el esfuerzo de búsqueda.
- Tiempo de ejecución: medido con `time.perf_counter()` inmediatamente antes y después de cada llamada, convertido a milisegundos porque en grafos pequeños los tiempos en segundos son casi cero.
- Calidad de la solución: se evalúa con dos criterios independientes — el costo real en metros (función `costo_ruta`) y el cumplimiento de la restricción de negocio (función `respeta_prioridad`). Una ruta más barata que viola la prioridad no es una solución válida para Ojo Camba.
- Se ejecutan dos instancias del problema (Secciones 5.2 y 5.3) para no basar la recomendación en un único caso favorable a algún algoritmo.

6.2. Caso 1 — lote típico de 5 reportes (Base → visitar R101...R105)

Algoritmo	Costo (m)	Nodos expl.	Tiempo (ms)	¿Respeta prioridad?	Ruta encontrada
BFS	4 250	207	0.2224	Sí	Base→R101→R102→R103→R104→R105
DFS	5 650	6	0.0128	No	Base→R105→R104→R103→R102→R101
Backtracking	4 250	9	0.1566	Sí	Base→R101→R102→R103→R104→R105

Interpretación: DFS es el más rápido (6 nodos explorados) pero no evalúa costo ni prioridad — encontró una ruta 33% más cara (5 650 m) y dejó la Emergencia para el final, violando la regla de negocio. BFS coincidió con el óptimo (4 250 m) en esta corrida, pero solo porque el orden fijo en que itera los vecinos coincidió con el orden de menor costo — no porque evalúe costos —, y para lograrlo tuvo que explorar 207 estados. Backtracking es el único que garantiza el óptimo por diseño (compara costos y poda) y lo hace explorando solo 9 estados, 23 veces menos que BFS.

6.3. Caso 2 — contraprueba con orden de exploración adverso

Para no basar la recomendación en la coincidencia favorable de la Sección 5.2, se repite el experimento con un segundo lote de 4 reportes (R201 Media, R202 Emergencia, R203 Alta, R204 Baja) cuya lista de iteración interna deja la Emergencia en segundo lugar. Esto obliga a BFS y DFS —que no aplican la restricción de prioridad— a exponer su punto débil:

Algoritmo	Costo (m)	Nodos expl.	Tiempo (ms)	¿Respeta prioridad?	Ruta encontrada
BFS	4 350	42	0.0687	No	Base→R201→R202→R203→R204
DFS	5 150	5	0.0098	No	Base→R204→R203→R202→R201
Backtracking	5 950	5	0.1728	Sí	Base→R202→R203→R201→R204

Interpretación: aquí BFS y DFS encuentran rutas numéricamente más baratas (4 350 m y 5 150 m) que backtracking (5 950 m), pero ambas violan la prioridad de gravedad: visitan la Media (R201) antes que la Emergencia (R202). Para Ojo Camba esas dos rutas no son soluciones utilizables, por más baratas que parezcan, porque retrasan la atención de una emergencia real. Backtracking es,

otra vez, el único algoritmo que entrega una solución válida y, dentro de las válidas, la de menor costo garantizado.

6.4. El “precio de la urgencia”

Comparando el óptimo libre (sin restricción) contra el óptimo legal (respetando la prioridad) en ambas instancias se obtiene una métrica útil para la discusión con el negocio: en el Caso 1 ambos coinciden en 4 250 m (la urgencia no costó nada extra), pero en el Caso 2 el óptimo libre es 4 350 m y el legal es 5 950 m — respetar la prioridad cuesta 1 600 m adicionales (37% más). Ese sobre costo no es un defecto del algoritmo: es el costo cuantificado de una decisión de diseño (atender primero lo urgente), y depende de la distribución geográfica de los reportes en cada lote.

6.5. Recomendación final fundamentada

Algoritmo recomendado para Ojo Camba: Backtracking con poda por costo y por prioridad de gravedad. La justificación combina tres criterios técnicos: (1) el negocio necesita minimizar metros reales, no saltos ni orden de iteración, lo que descarta a BFS como criterio único de optimalidad — la Sección 5.3 muestra que puede coincidir con el óptimo por azar o desviarse por completo; (2) el negocio necesita una garantía de validez y de optimalidad verificable, no una respuesta que “a veces” es la mejor y que además puede infringir la regla de atención por gravedad, lo que descarta a DFS; (3) el tamaño real de un lote de reportes por técnico es pequeño (decenas, no miles), por lo que el mayor número de estados que explora backtracking frente a DFS sigue siendo del orden de milisegundos y no representa ningún costo computacional relevante.

Trabajo futuro (fuera del alcance algorítmico de esta actividad): si Ojo Camba debe asignar reportes entre n técnicos simultáneamente, el problema se convierte en un Vehicle Routing Problem (VRP); backtracking exhaustivo deja de ser práctico por su crecimiento combinatorio ($n!$ por técnico) y correspondería migrar a heurísticas (vecino más cercano, algoritmos genéticos, OR-Tools de Google) o a búsqueda informada con A^* (Hart, Nilsson y Raphael, 1968), que combina costo real y heurística de distancia restante para explorar muchos menos nodos que un backtracking exhaustivo.

7. Auditoría de IA

Bitácora de uso y transparencia de IA, completada desde el primer día de trabajo, con registro de al menos un caso Descartado (fila 3) y evidencia de verificación humana independiente del código generado (fila 5):

Fecha	Fase	Herramienta	Prompt (resumen)	Sugerencia obtenida	Decisión	Verificación
Día 1	Selección del caso	Asistente de IA (chat)	Cómo extraer un grafo de OpenStreetMap para optimizar rutas	Sugirió OSMnx + NetworkX y explicó nodos/aristas/costo	Aceptado	Se contrastó con la documentación oficial de OSMnx
Día 1	Formalización	Asistente de IA (chat)	Cómo definir estado/meta/operadores para un técnico que visita varios reportes	Formalización con estado=(posición,visitados) ; meta=condición, no lugar	Aceptado	Verificado a mano con el caso de 2 reportes (Sección 5.1)
Día 2	Datos del caso	Asistente de IA (chat)	Cómo obtener tiempos de viaje reales entre puntos de Santa Cruz	Sugirió inicialmente una API de pago (Google Maps Distance Matrix)	Descartado	Se reemplazó por distancias estimadas por el equipo, documentadas como no oficiales
Día 3	Implementación	Asistente de IA (código)	Generar visualizador interactivo de BFS/DFS/backtracking	Primera versión resolvía llegar a UN reporte, no visitar todos	Modificado	Se exigió rehacer con estado=(posición, conjunto de visitados)
Día 3	Verificación	Auditoría propia (sin IA)	Cálculo manual del costo de las rutas legales del caso Base-E1-E2	N/A (verificación humana)	Validado	Coincide con la salida del código (Sección 5.1)
Día 4	Análisis	Asistente de IA (chat)	Por qué el botón de prioridad de Emergencia daba resultados inconsistentes	Detectó que la regla solo se aplicaba en un algoritmo, no en los tres	Corregido	Se unificó la regla y se validó con el caso B/E1/E2 a mano
Día 4	Análisis	Asistente de IA (código)	Ejecutar BFS/DFS/backtracking sobre el lote de 5 reportes y comparar	Código funcional con contadores de nodos y tiempo	Aceptado y ejecutado	Salida real verificada línea por línea (Sección 5.2)

7.1. Auditoría crítica destacada

Situación: la primera versión del visualizador interactivo, generada con apoyo de IA, resolvía únicamente “llegar a un solo reporte” (búsqueda punto a punto), no “visitar todos los reportes del lote”, que es el problema real que enfrenta el técnico de Ojo Camba.

Decisión crítica tomada: se rechazó esa formalización y se exigió rehacerla con estado = (posición, conjunto de reportes visitados), meta = todos visitados, siguiendo el razonamiento del Paso 3 de este informe.

Justificación: aceptar sin cuestionar una simplificación que cambia el problema de fondo es exactamente el tipo de uso acrítico que la rúbrica institucional busca detectar en el Criterio 4. Verificar esta condición antes de aceptar la sugerencia — y, más adelante, diseñar la contraprueba de la Sección 5.3 para exponer que BFS/DFS pueden violar la regla de negocio — es lo que sostiene la validez técnica de este informe.

Un segundo hallazgo de auditoría: al pedir ayuda para calcular distancias reales entre puntos de Santa Cruz de la Sierra, la primera sugerencia de la IA fue usar la API de pago Google Maps Distance Matrix. Se descartó por introducir una dependencia de pago no declarada, y se reemplazó por (a) OpenStreetMap + Overpass API como opción gratuita para producción, o (b) estimaciones propias del equipo, documentadas explícitamente como no oficiales — la opción utilizada en este informe (Sección 1.2).

8. Conclusiones

El problema de ruteo del técnico de Ojo Camba se formaliza correctamente como un espacio de estados discreto, determinista, estático, totalmente observable y secuencial, resoluble mediante búsqueda simbólica sin necesidad de aprendizaje estadístico.

BFS y DFS son insuficientes por sí solos para este caso: ninguno evalúa costo real ni la restricción de prioridad por gravedad, y ambos pueden entregar soluciones inválidas o subóptimas según el orden de iteración del grafo, como lo demuestra la contraprueba del Paso 5.3.

Backtracking con poda por costo y por prioridad es el algoritmo recomendado: garantiza simultáneamente el costo mínimo y el cumplimiento de la regla de negocio, explorando una fracción de los estados que requiere BFS.

El “precio de la urgencia” (Sección 5.4) es una métrica reutilizable para que Ojo Camba cuantifique, lote a lote, cuánto cuesta en metros priorizar las emergencias — información valiosa para decisiones de negocio, no solo técnicas.

9. Referencias

Russell, S. y Norvig, P. *Inteligencia Artificial: un enfoque moderno*. Pearson.

Newell, A. y Simon, H. (1956). *The Logic Theory Machine: A complex information processing system*.

Newell, A. y Simon, H. (1957). *General Problem Solver: A program that simulates human thought*.

Hart, P., Nilsson, N. y Raphael, B. (1968). *A Formal Basis for the Heuristic Determination of Minimum Cost Paths*. *IEEE Transactions on Systems Science and Cybernetics*.

Repositorio del proyecto Ojo Camba: github.com/jhoel0521/ojo-camba